

DOPPL — the agent that builds the agents that build the agents

A self-replicating idea organism. An Idearorganism. A population of agents under selection pressure that evolves toward generating non-obvious, verifiable ideas — by transplanting techniques across fields and reading the zeitgeist.

The limit of A/B testing. Not A vs. B, not even A through Z — an open, self-extending alphabet of variants no human wrote, where winning means surviving adversarial scrutiny, and each generation breeds the next. **Ideas By No One, For No One¹.** A/Z testing finds your least-bad idea; Doppl evolves one worth having.

Direction: B (philosophically different agent) + A (classical ML governs the population) · Team: 3–4 · Showcase: Jun 29

The Problem — and why it's hard

Every agent system shipped today is a **hand-built artifact**: a human designs the prompt, the toolset, the decomposition, and the verification loop, then the agent executes that fixed design. The intelligence lives in the *scaffold*, and the scaffold is frozen at the moment a human stops typing. We are asking: **what if the scaffold itself were under selection pressure?** Instead of designing one agent, design the *evolutionary dynamics* of a population of agents, give it a fixed compute budget as its environment, and let competition discover scaffolds no human would have written.

We point this organism at the hardest thing to automate: **having a genuinely good idea**. Concretely, two prey: (1) **cross-domain transfer** — finding a technique or result in field A that cracks an open problem in field B (the move behind most real breakthroughs); and (2) **zeitgeist synthesis** — surfacing a thesis, product, or framing that fits the present moment and survives scrutiny. This is hard for the deep reason that **“good idea” has no cheap ground-truth signal**. You cannot grade novelty with a unit test. So the central technical bet is to *manufacture a fitness function out of adversarial verification* and use evolution to climb it without collapsing into confident slop.

Technical Approach — AI as the primitive, not the bolt-on

The unit of life is an **agent genome, or Agenome**: a serialized {system-prompt + persona/value-weights + tool permissions + decomposition policy + spawn budget}. The organism is a generational loop over a population of genomes.

- **Metabolism.** Each agent receives a finite token/compute allowance — its energy. Reasoning, tool calls, and spawning all *cost* energy. An agent that produces verified value is *rewarded with more*; one that burns budget on nothing starves. Compute scarcity is the environment that makes selection real, not cosmetic.
- **Reproduction by Fusion² — two parents, not one.** Most agent systems reproduce asexually (one parent → a mutated copy), which converges to a local optimum. Doppl reproduces *sexually*: two high-fitness parent agenomes **fuse** into offspring that inherits from both, then mutates — exactly why sexual reproduction exists in nature: recombining good traits across lineages escapes local optima far faster than mutation alone. Fusion ships at two levels — *agenome-level crossover* (splice two parents' prompts, personas, toolsets) and *output-level fusion* (a model synthesizes two parents' reasoning into a child). The rhyme is the point: both prey are themselves fusions of two things from different domains, so an organism whose reproduction *is* fusion is structurally aimed at the ideas we want. This is the recursive engine: agents author the next generation of agents.
- **Selection via an adversarial critic council.** Every candidate idea faces a spawned panel of red-team critics with distinct mandates (factual grounding, novelty/prior-art, feasibility, falsification). An idea reproduces only if it survives the council *and* any available real-world check (the transfer actually runs on a test case; a claim resolves against data). The council *is* the fitness function — the “argue-with-yourself” Council, repurposed as evolutionary pressure.
- **Classical ML governs the ecosystem (Direction A).** Spawn-budget allocation is a *multi-armed bandit / RL control problem*: which lineages deserve more compute? We learn an embedding of “idea space,” run diversity/novelty scoring (nearest-neighbor distance, determinantal point processes) to prevent mode collapse, and use a learned value model to predict which agenomes are worth feeding before spending tokens. The LLMs supply generativity; classical ML supplies **control, credit assignment, and search**.
- **Recursive ideation + a fitness function that evolves itself (the apex bet).** A human supplies only the seed — the initial impulse. From there the organism recursively ideates: ideas spawn sub-ideas, agents spawn sub-agents, generation feeds generation. Crucially, the *critic council is itself under selection*, so the organism doesn't just search for better ideas — it searches for **better definitions of “better.”** The hard rail: a self-optimized metric invites reward hacking (“redefine winning as whatever I already produce”), so metric mutations survive only if they keep correlating with bedrock we can't fake — executable checks, real-world resolution, held-out human judgment. The objective can evolve; its anchor cannot move.
- **De-risked starting agenome (we already have the seed).** Our generation-0 organism is not hypothetical: it's an existing, working skill — *Rule of Cool* — that absorbs context, silently generates and ranks cross-domain candidates (what a product designer / novelist / veteran engineer would each add), applies a quality filter, and emits the single best surviving idea. That is the single-agent, human-authored, frozen version of exactly one Doppl agenome. The capstone is taking that proven scaffold, **mutating it into a population and putting it under selection** — one good idea-machine becoming an ecosystem that breeds better ones.

Scope — what “working” means in two weeks

Minimum shippable cut	Ambitious version (the moonshot)
A single-generation loop on a fixed problem set: spawn ~20 agenomes, reproduce by Fusion (crossover + output synthesis), run the critic council, cull, mutate, re-run. Show generation N+1 measurably beats generation N on a held-out idea-quality rubric.	Multi-generational, open-ended population with live compute economy, learned spawn-allocation, and novelty pressure — running for hours and producing emergent lineage specialization (a “transfer-hunter” species, a “contrarian” species).
Verification = critic council + one objective check per domain (e.g., a cross-domain transfer we can actually execute and score pass/fail).	The in-house flywheel (the moonshot). Every run yields a labeled dataset of idea-making behavior; we distill those meta-lessons into weights — fine-tuning our own in-house model on the winning lineages, so it gets permanently better at having ideas. Open-weight models also unlock weight-level fusion (merging parent models, not just outputs). The slow loop wrapped around the fast loop.
Instrumented dashboard: population tree, energy per agent, fitness over generations.	Self-improving verifier + live, watchable evolution: critics themselves evolve and the population visibly reorganizes on screen as selection runs.

Why it's compelling — the recruiting hook

Crack engineers don't want to wire another SaaS pipeline out of existing APIs; they want to build a **kernel**. Doppl is an agent operating system where the agents breed the agents — population dynamics, sexual reproduction by fusion, a compute economy, learned credit assignment, an evolving verifier, and a model that fine-tunes itself on its own winners. It touches RL, search, multi-agent systems, model merging, and artificial life at once, and the demo is unforgettable: **you watch a digital ecosystem get smarter in real time**. It is the literal embodiment of the course thesis — AI as a core primitive — because there is no fixed app here, only a process that *manufactures* the app. **“It's not the agent — it's the agents that breed the agents.”**

Hard parts / risks — the bet, and where it breaks

- **Fitness without ground truth (the core bet).** If the critic council is a weak judge, evolution optimizes for *fooling the critics*, not for good ideas. Mitigation: ground critics in retrieval + executable checks, hold out judges from the breeding loop, and rotate critic agenomes so the target keeps moving.
- **Mode collapse / slop convergence.** Populations love to converge on one safe, mediocre genome. Mitigation: explicit novelty pressure (DPP/quality-diversity, MAP-Elites-style niches); Fusion across distant lineages is itself an anti-collapse force.
- **Cost & termination.** Recursive spawning can blow the budget combinatorially. The metabolism is the safety rail: hard energy caps, depth limits, and the bandit allocator keep the tree finite — a feature we demo, not just a constraint.
- **Two-week realism.** The moonshot may not fully converge in 14 days — the brief rewards *ambition plus how much you got working*. The single-generation Fusion cut is guaranteed to run and show the improvement effect; the self-evolving verifier and in-house fine-tuning flywheel are explicitly stretch, not MVP.

What we'd demo — the live 10 minutes (Jun 29)

(1) **Seed:** we hand the organism a live, unsolved prompt from the room. (2) **Watch it live:** the population tree renders on screen — agents spawn, spend energy, ideas hit the critic council, weak lineages go dark, strong pairs fuse into offspring and mutate. (3) **Generations climb:** a fitness-over-time chart rises as later generations beat earlier ones; lineages visibly specialize. (4) **Payoff:** the organism surfaces its best surviving idea, replays the adversarial gauntlet it passed, and — for transfer prompts — we *execute the proposed transfer live*. The contrast we hammer: this isn't a tournament where variants win or die in one round — it's **generational**. **The winners breed, two parents fuse, the losers' useful fragments get spliced back in, and round N+1 is genuinely smarter than round N.** One sentence: **we didn't build the idea — we built the thing that evolved it.**

Team — who owns what (3–4 engineers)

Owner	Surface
Kernel / runtime	Agenome schema, the generational loop, the metabolism (energy accounting), Fusion reproduction (crossover + output synthesis), spawn/cull mechanics, depth & budget caps. The substrate everything else plugs into.
Selection / ML	Bandit + value-model spawn allocation, idea-space embeddings, novelty/quality-diversity scoring, fitness tracking and credit assignment; owns the moonshot in-house fine-tuning flywheel. Direction-A core.
Verifier council	Adversarial critic agenomes, retrieval grounding, executable transfer checks, anti-reward-hacking (held-out + rotating judges). Owns the fitness signal's integrity.
Demo / observability	Live population-tree visualization, fitness-over-time charts, energy telemetry, and the Jun-29 demo harness. Owns what the room sees.

A 3-person team merges Kernel and Demo under one owner; a 4th lets each surface run in parallel from day one. Week 1: kernel + single-generation Fusion loop end-to-end. Week 2: turn on the compute economy, learned allocation, novelty pressure, the live visualization, and — if the inner loop holds — the in-house fine-tuning flywheel.